

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import MinMaxScaler
from sklearn.cluster import KMeans
from sklearn import metrics
from collections import Counter
from matplotlib.colors import ListedColormap
```

```
In [2]: df = pd.read_csv(r"C:\Users\admin\Downloads\Orders1.csv")
df
```

```
Out[2]:
```

	Order_ID	Product_Category	Order_Amount_USD	Shipping_Distance_KM	Delivery_I
0	ORD-50001	Apparel	134.14	141.3	
1	ORD-50002	Home & Kitchen	116.17	50.0	
2	ORD-50003	Electronics	346.13	178.0	
3	ORD-50004	Electronics	742.90	194.1	
4	ORD-50005	Home & Kitchen	361.26	50.0	
...	...	...	...	...	...
495	ORD-50496	Electronics	808.43	1500.0	
496	ORD-50497	Books	11.42	1353.6	
497	ORD-50498	Apparel	165.49	1461.0	
498	ORD-50499	Home & Kitchen	198.03	1347.9	
499	ORD-50500	Electronics	849.89	1385.3	

500 rows × 7 columns



```
In [3]: df.drop(['Order_Amount_USD', 'Shipping_Distance_KM'], axis=1)

data = df.drop(['Order_ID', 'Product_Category', 'Satisfied_Customer', 'Return_Reque
data
```

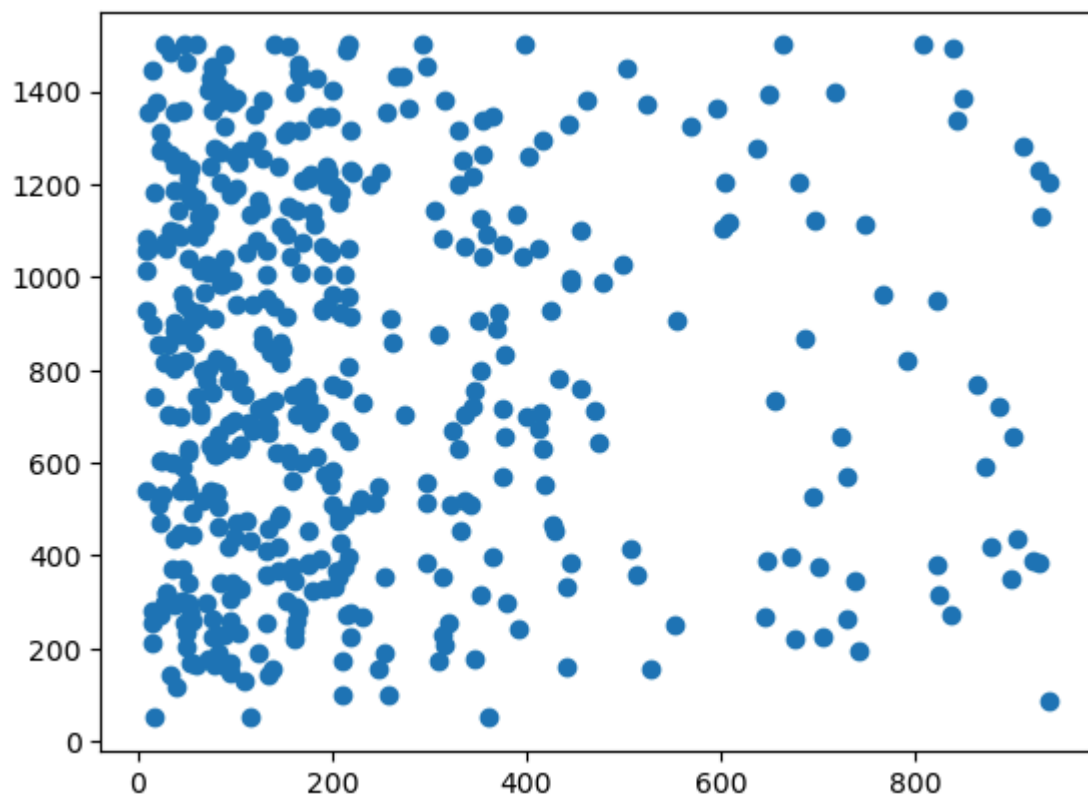
Out[3]:

	Order_Amount_USD	Shipping_Distance_KM	Delivery_Days
0	134.14	141.3	3.16
1	116.17	50.0	2.55
2	346.13	178.0	1.93
3	742.90	194.1	1.47
4	361.26	50.0	1.00
...	...	...	...
495	808.43	1500.0	7.07
496	11.42	1353.6	6.85
497	165.49	1461.0	6.79
498	198.03	1347.9	6.42
499	849.89	1385.3	6.99

500 rows × 3 columns

```
In [4]: plt.scatter(data.Order_Amount_USD, data.Shipping_Distance_KM)
```

Out[4]: <matplotlib.collections.PathCollection at 0x26a59958ad0>



```
In [6]: S = MinMaxScaler()
data = S.fit_transform(data)
data
```

```
Out[6]: array([[0.13561347, 0.06296552, 0.31578947],
               [0.1162678 , 0.          , 0.22660819],
               [0.3638321 , 0.08827586, 0.13596491],
               ...,
               [0.16936343, 0.97310345, 0.84649123],
               [0.20439449, 0.89510345, 0.79239766],
               [0.90615681, 0.92089655, 0.87573099]])
```

```
In [7]: data = pd.DataFrame(data, columns=['A', 'B', 'C'])
data
```

```
Out[7]:
```

	A	B	C
0	0.135613	0.062966	0.315789
1	0.116268	0.000000	0.226608
2	0.363832	0.088276	0.135965
3	0.790976	0.099379	0.068713
4	0.380120	0.000000	0.000000
...	...	...	...
495	0.861523	1.000000	0.887427
496	0.003499	0.899034	0.855263
497	0.169363	0.973103	0.846491
498	0.204394	0.895103	0.792398
499	0.906157	0.920897	0.875731

500 rows × 3 columns

```
In [8]: model = KMeans(n_clusters=2)
model.fit(data)
```

C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.

warnings.warn(

```
Out[8]:
```

▼
KMeans
i ?

KMeans(n_clusters=2)

```
In [9]: y = model.fit_predict(data)
y
```

C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.

warnings.warn(

[illegible]

```
In [10]: data['cluster'] = y
data
```

Out[10]:	A	B	C	cluster
0	0.135613	0.062966	0.315789	1
1	0.116268	0.000000	0.226608	1
2	0.363832	0.088276	0.135965	1
3	0.790976	0.099379	0.068713	1
4	0.380120	0.000000	0.000000	1
...	...	...	...	...
495	0.861523	1.000000	0.887427	0
496	0.003499	0.899034	0.855263	0
497	0.169363	0.973103	0.846491	0
498	0.204394	0.895103	0.792398	0
499	0.906157	0.920897	0.875731	0

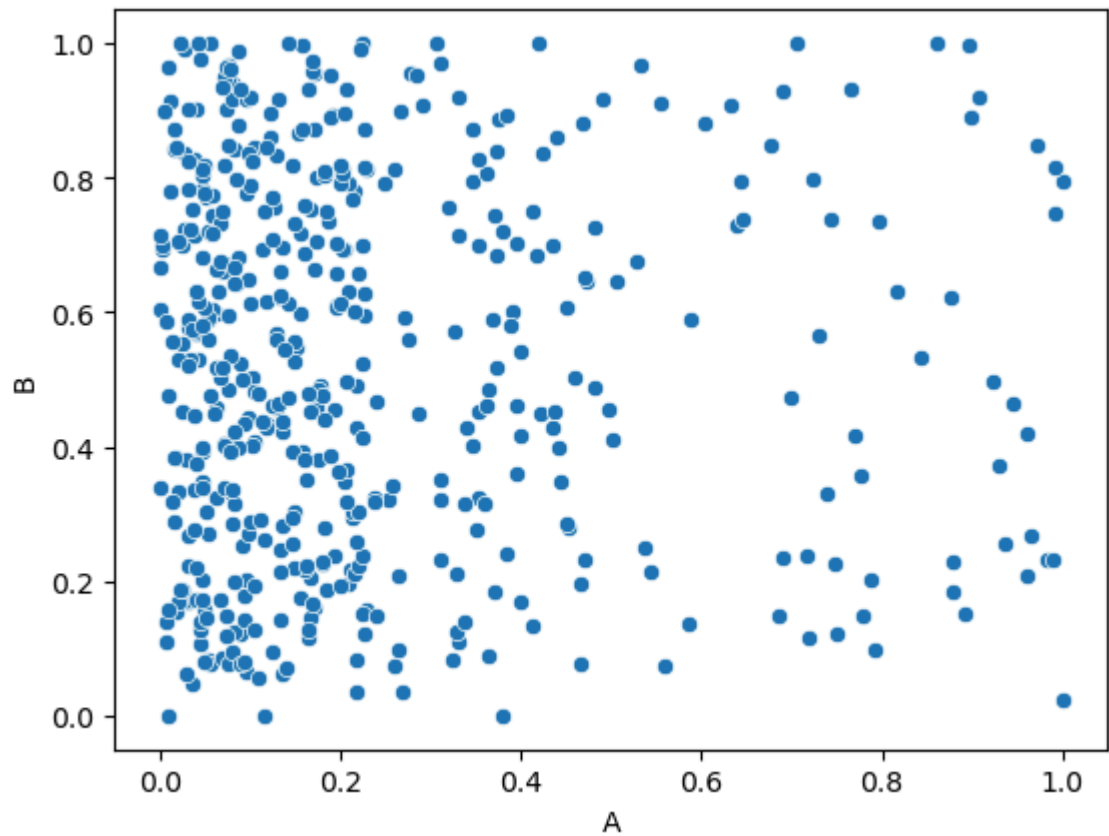
500 rows $\times$ 4 columns

```
In [11]: cent = model.cluster_centers_
          cent
```

```
Out[11]: array([[0.21756884, 0.75406978, 0.68730718],
                [0.25119999, 0.27774769, 0.32238031]])
```

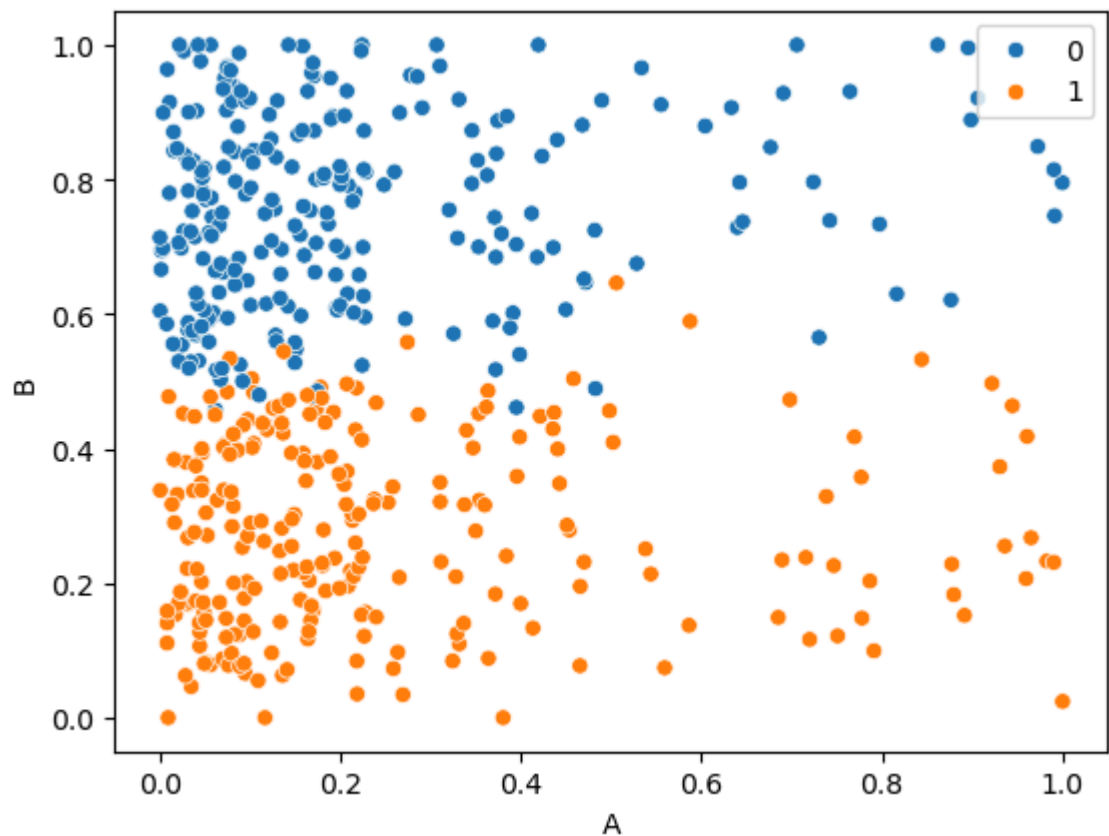
```
In [12]: Counter(model.labels_)
sns.scatterplot(data=data, x='A', y='B')
```

Out[12]: <Axes: xlabel='A', ylabel='B'>



```
In [13]: sns.scatterplot(data=data, x='A', y='B', hue=model.labels_)
```

Out[13]: <Axes: xlabel='A', ylabel='B'>



```
In [14]: K_rng = range(1,11)
         Sse = []
```

```
for K in K_rng:
    km = KMeans(n_clusters=K)
    km.fit(data[['A', 'B']])
    Sse.append(km.inertia_)
```

Sse

C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.

warnings.warn(

C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.

warnings.warn(

C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.

warnings.warn(

C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.

warnings.warn(

C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.

warnings.warn(

C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.

warnings.warn(

C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.

warnings.warn(

C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.

warnings.warn(

C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.

warnings.warn(

C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.

warnings.warn(

```
Out[14]: [66.24574598022315,
46.03790203192374,
21.6577441649115,
17.10602224706561,
12.591770065122942,
10.284607812239503,
8.492499360585164,
7.564131347055051,
6.295489384990929,
6.178984212756489]
```

```
In [15]: df
```

```
Out[15]:
```

	Order_ID	Product_Category	Order_Amount_USD	Shipping_Distance_KM	Delivery_I
--	----------	------------------	------------------	----------------------	------------

0	ORD-50001	Apparel	134.14	141.3	
1	ORD-50002	Home & Kitchen	116.17	50.0	
2	ORD-50003	Electronics	346.13	178.0	
3	ORD-50004	Electronics	742.90	194.1	
4	ORD-50005	Home & Kitchen	361.26	50.0	
...	...	...	...	...	
495	ORD-50496	Electronics	808.43	1500.0	
496	ORD-50497	Books	11.42	1353.6	
497	ORD-50498	Apparel	165.49	1461.0	
498	ORD-50499	Home & Kitchen	198.03	1347.9	
499	ORD-50500	Electronics	849.89	1385.3	

500 rows × 7 columns



```
In [16]: rng = np.random.default_rng(42)

new_rows = []
for i in range(20):
    new_rows.append({
        'Order_ID': f'ORD-9{i+1:04d}',
        'Product_Category': rng.choice(df['Product_Category'].unique()),
        'Order_Amount_USD': round(float(rng.uniform(80, 900)), 2),
        'Shipping_Distance_KM': round(float(rng.uniform(30, 2200)), 1),
        'Delivery_Days': round(float(rng.uniform(1, 12)), 2),
        'Satisfied_Customer': rng.choice(['Yes', 'No']),
    })
```

```
        'Return_Requested': rng.choice(['Yes', 'No'])
    })

new_df = pd.DataFrame(new_rows)
df2 = pd.concat([df, new_df], ignore_index=True)
df2.tail(20)
```

Out[16]:

	Order_ID	Product_Category	Order_Amount_USD	Shipping_Distance_KM	Delivery_I
--	----------	------------------	------------------	----------------------	------------

500	ORD-90001	Apparel	439.88	1893.2	
501	ORD-90002	Apparel	880.01	1681.7	
502	ORD-90003	Sports & Outdoors	384.05	2041.1	
503	ORD-90004	Sports & Outdoors	443.60	523.1	
504	ORD-90005	Sports & Outdoors	597.96	1675.1	
505	ORD-90006	Sports & Outdoors	812.36	1719.1	
506	ORD-90007	Electronics	206.52	1512.2	
507	ORD-90008	Sports & Outdoors	347.18	833.9	
508	ORD-90009	Electronics	470.08	522.4	
509	ORD-90010	Electronics	762.80	1549.6	
510	ORD-90011	Electronics	397.73	655.7	
511	ORD-90012	Apparel	243.92	46.0	
512	ORD-90013	Electronics	720.20	1025.8	
513	ORD-90014	Apparel	173.91	1480.4	
514	ORD-90015	Apparel	600.47	1231.3	
515	ORD-90016	Home & Kitchen	105.27	977.7	
516	ORD-90017	Sports & Outdoors	271.83	156.5	
517	ORD-90018	Home & Kitchen	622.77	1238.8	
518	ORD-90019	Electronics	747.50	392.3	
519	ORD-90020	Apparel	672.33	1032.3	

```
In [17]: feats = ['Order_Amount_USD', 'Shipping_Distance_KM', 'Delivery_Days']

scaler2 = MinMaxScaler()
data2 = pd.DataFrame(scaler2.fit_transform(df2[feats]), columns=['A', 'B', 'C'])
data2.head()
```

```
Out[17]:
```

	A	B	C
0	0.135613	0.047767	0.249423
1	0.116268	0.002005	0.178984
2	0.363832	0.066162	0.107390
3	0.790976	0.074232	0.054273
4	0.380120	0.002005	0.000000

```
In [18]: model2 = KMeans(n_clusters=2, n_init=10, random_state=0)
model2.fit(data2[['A', 'B', 'C']])
Counter(model2.labels_)
```

C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=3.

```
warnings.warn(
```

```
Out[18]: Counter({np.int32(1): 264, np.int32(0): 256})
```

```
In [19]: print("Old inertia:", model.inertia_)
print("New inertia:", model2.inertia_)

print("\nOld centroids (scaled):\n", model.cluster_centers_)
print("\nNew centroids (scaled):\n", model2.cluster_centers_)
```

```
Old inertia: 45.798016652013224
New inertia: 43.75882766728799
```

```
Old centroids (scaled):
[[0.21756884 0.75406978 0.68730718]
 [0.25119999 0.27774769 0.32238031]]
```

```
New centroids (scaled):
[[0.25430579 0.21012457 0.25656755]
 [0.2365067 0.55580206 0.558699  ]]
```

```
In [21]: scaler = MinMaxScaler()
data = pd.DataFrame(scaler.fit_transform(df[feats]), columns=['A', 'B', 'C'])

model = KMeans(n_clusters=2, n_init=10, random_state=0)
model.fit(data[['A', 'B', 'C']])

old_real = pd.DataFrame(scaler.inverse_transform(model.cluster_centers_), columns=feats)
new_real = pd.DataFrame(scaler2.inverse_transform(model2.cluster_centers_), columns=feats)

print("Old centroids (real units):\n", old_real.round(2))
print("\nNew centroids (real units):\n", new_real.round(2))
```

Old centroids (real units):

	Order_Amount_USD	Shipping_Distance_KM	Delivery_Days
0	241.51	452.73	3.21
1	210.27	1143.40	5.70

New centroids (real units):

	Order_Amount_USD	Shipping_Distance_KM	Delivery_Days
0	244.39	465.22	3.22
1	227.86	1154.88	5.84

C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.

warnings.warn(

```
In [22]: df['Cluster'] = model.labels_  
df2['Cluster'] = model2.labels_  
  
print(df.groupby('Cluster')[feats].agg(['mean', 'count']).round(2))  
print(df2.groupby('Cluster')[feats].agg(['mean', 'count']).round(2))
```

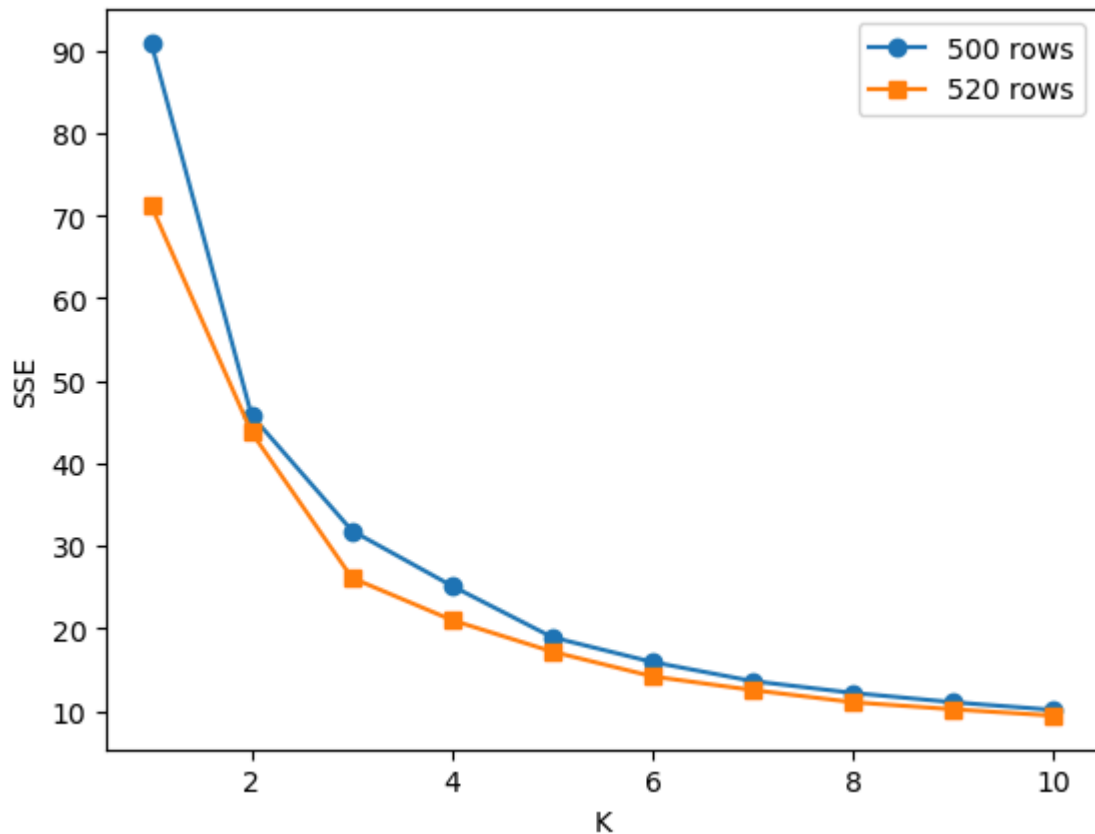
	Order_Amount_USD		Shipping_Distance_KM		Delivery_Days	
	mean	count	mean	count	mean	count
Cluster						
0	241.51	246	452.73	246	3.21	246
1	210.27	254	1143.40	254	5.70	254

	Order_Amount_USD		Shipping_Distance_KM		Delivery_Days	
	mean	count	mean	count	mean	count
Cluster						
0	244.39	256	465.22	256	3.22	256
1	227.86	264	1154.88	264	5.84	264

```
In [23]: Sse_old, Sse_new = [], []  
for K in range(1, 11):  
    Sse_old.append(KMeans(n_clusters=K, n_init=10, random_state=0).fit(data[['A'  
    Sse_new.append(KMeans(n_clusters=K, n_init=10, random_state=0).fit(data2[['A'  
  
plt.plot(range(1,11), Sse_old, marker='o', label='500 rows')  
plt.plot(range(1,11), Sse_new, marker='s', label='520 rows')  
plt.xlabel('K'); plt.ylabel('SSE'); plt.legend(); plt.show()
```

[illegible]

```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.
  warnings.warn(
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=3.
  warnings.warn(
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.
  warnings.warn(
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=3.
  warnings.warn(
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.
  warnings.warn(
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=3.
  warnings.warn(
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.
  warnings.warn(
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=3.
  warnings.warn(
```

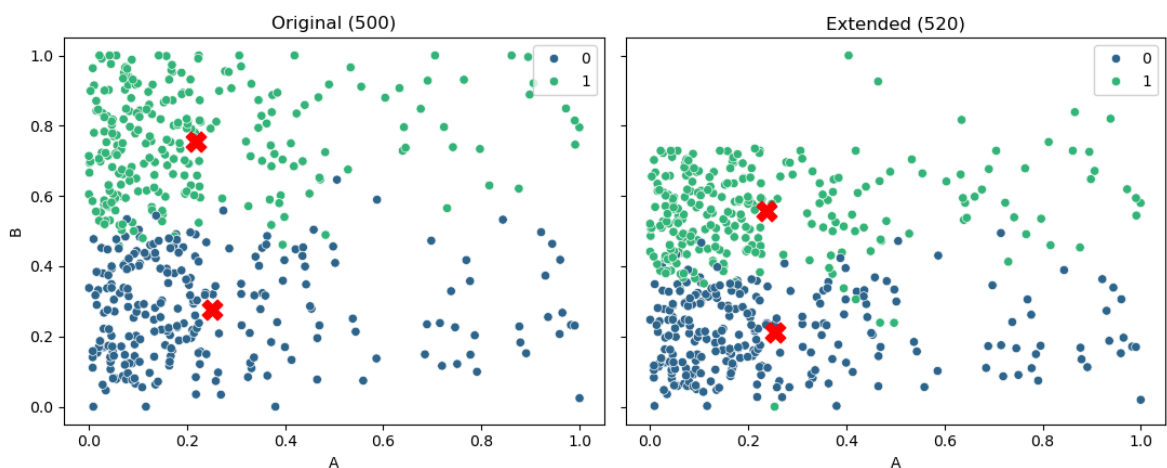


```
In [24]: fig, ax = plt.subplots(1, 2, figsize=(11, 4.5), sharex=True, sharey=True)

sns.scatterplot(data=data, x='A', y='B', hue=model.labels_, ax=ax[0], palette='v
ax[0].scatter(model.cluster_centers_[0], model.cluster_centers_[1], c='red',
ax[0].set_title('Original (500)')

sns.scatterplot(data=data2, x='A', y='B', hue=model2.labels_, ax=ax[1], palette=
ax[1].scatter(model2.cluster_centers_[0], model2.cluster_centers_[1], c='red'
ax[1].set_title('Extended (520)')

plt.tight_layout(); plt.show()
```



```
In [25]: df2.to_csv('Orders1_extended.csv', index=False)
```

```
In [ ]:
```